

# Code-Layout, Readability and Reusability

|               |                                                      |
|---------------|------------------------------------------------------|
| Date          | 12 July 2026                                         |
| Team ID       | 6a2fbf036b1ed0bd8fa789aa                             |
| Project Name  | AI Powered Debt Relief & Financial Recovery Platform |
| Maximum Marks | 5 Marks                                              |

## Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

### Code Layout Checklist:

| S.No | Code Quality Parameter    | Description                                   | Followed (Yes/No/Partial) | Remarks                                                                                                                       |
|------|---------------------------|-----------------------------------------------|---------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| 1    | Consistent Indentation    | Uniform spacing/tabs used throughout the code | Yes                       | PEP 8 enforced in backend (Black formatter); Prettier + ESLint used for React frontend.                                       |
| 2    | Proper File Structure     | Files and folders are logically organized     | Yes                       | Backend split into routers/, models/, services/, schemas/; frontend organized by components/, pages/, hooks/.                 |
| 3    | Meaningful Variable Names | Variables reflect their purpose clearly       | Yes                       | e.g., emi_ratio, debt_stress_score, settlement_amount instead of generic names.                                               |
| 4    | Function / Method Names   | Functions are descriptively named             | Yes                       | e.g., calculate_debt_stress(), generate_negotiation_letter(), predict_settlement_amount().                                    |
| 5    | Code Comments             | Inline and block comments explain logic       | Partial                   | Core financial calculation and AI integration modules are well commented; some minor UI components need more inline comments. |
| 6    | Modular Design            | Code is split into reusable functions/modules | Yes                       | Financial Health Engine, Debt Stress Analyzer, and Gemini integration are separate, independently testable services.          |
| 7    | No Redundant Code         | Duplicate or unused code is removed           | Yes                       | Shared logic (e.g., API request handling, JWT validation) extracted into common utility modules.                              |
| 8    | Error Handling            | Exceptions and errors are handled gracefully  | Yes                       | FastAPI exception handlers return structured error responses; Gemini API failures trigger rule-based fallback logic.          |

**Reusable Components / Modules:**

| S.No | Component / Module Name                                        | Language / Technology | Where Reused                                                          | Reusability Level (High/Medium/Low) |
|------|----------------------------------------------------------------|-----------------------|-----------------------------------------------------------------------|-------------------------------------|
| 1    | JWT Authentication Middleware                                  | Python (FastAPI)      | All protected API routes (loans, settlements, negotiation, dashboard) | High                                |
| 2    | Financial Calculation Utilities (EMI ratio, debt stress score) | Python                | Financial Dashboard API, Settlement Prediction Engine                 | High                                |
| 3    | Gemini API Client Wrapper                                      | Python                | AI Negotiation Letter Generator, AI Negotiation Strategy module       | High                                |
| 4    | Reusable UI Components (Card, Modal, Chart, Button)            | React.js              | Financial Dashboard, Settlement Predictor, AI History Module          | Medium                              |
| 5    | Axios API Service Layer                                        | JavaScript (React)    | All frontend modules communicating with the FastAPI backend           | High                                |

**Overall Code Quality Assessment:**

| Aspect                   | Rating (1-5) | Comments                                                                                                         |
|--------------------------|--------------|------------------------------------------------------------------------------------------------------------------|
| Code Layout & Structure  | 5            | Clear separation of routers, models, schemas, and services following FastAPI best practices.                     |
| Readability              | 4            | Consistent naming and formatting; a few UI components could use additional inline comments.                      |
| Reusability              | 5            | Core financial logic and AI integration are modular and reused across multiple endpoints and views.              |
| Documentation / Comments | 4            | Key modules are well documented; API documentation auto-generated via FastAPI's OpenAPI/Swagger.                 |
| Overall Score            | 4.5          | Well-structured, maintainable codebase with strong modularity and minor room for improved in-line documentation. |